---
title: EditSpace
subtitle: Shared 3D Space System and Swift API
author: Graphite 3D
date: September 2026

Purpose

EditSpace is a language-neutral collaboration protocol for shared 3D spaces. A space is the synchronized scene, including objects, transforms, geometry, materials, modifiers, assets, hierarchy, and presence. The specification is the source of truth. This repository contains the Swift reference implementation.

The design separates durable scene edits from ephemeral human presence. An endpoint can render other collaborators, their selections, cursors, rays, cameras, or active tools without placing that transient state in the space history.

Design rules

1. Operations are truth and snapshots are caches.
2. Every accepted operation is immutable and globally identified.
3. App and renderer types never cross the protocol boundary.
4. Causal order is honored; concurrent ties use `(sequence, actor ID, operation ID)`.
5. Fields merge independently as last-writer-wins registers.
6. Deletion is represented by a tombstone.
7. Unknown future operations are preserved and reported as compatibility data.
8. Identity and presence are first-class protocol messages but never durable scene edits.

Components

The specification defines the common wire contract and convergence rules. The Swift and Python libraries descend independently from that specification and exchange the same messages. GraphiteKit imports EditSpace Swift, and Graphite 3D uses GraphiteKit as its modelling and collaboration library. The Blender plug-in imports EditSpace Python and is hosted by Blender. Graphite 3D users and Blender users interact with their respective applications rather than with the protocol libraries directly. Within either language library, the endpoint adapter converts native actions and values into EditSpace operations. `OperationLog` validates, deduplicates, and preserves them. `Materializer` deterministically reduces the log into `SpaceState`. `Replica` keeps that state private and projects winning mutations through application-owned functions. `SyncEngine` manages peer and durable-store queues while leaving networking and storage to adapter protocols.

Wire protocol

Version 1 messages are UTF-8 JSON. The durable envelope kind is `editspace.operations`; the ephemeral kind is `editspace.presence`. Dates use RFC 3339/ISO 8601 UTC strings. Values use only the JSON value domain.

Durable operation

An operation includes schema version, shared-space ID (encoded as `doc` for v1 compatibility), immutable operation ID, actor ID, actor-local sequence, causal dependencies, action, scene entity kind, target/source IDs, typed 3D field writes and arguments, required feature tokens, and optional producer diagnostics.

Core actions are create, update, delete, duplicate, link, and unlink. Core entities cover spaces, objects, meshes, vertices, faces, modifiers, materials, assets, constraints, parameters, dependencies, and legacy snapshots. The field vocabulary specifies transforms, mesh topology, modifier inputs, PBR values, and asset references rather than treating fields as application-private data.

Exact duplicates are ignored. Reusing an ID with different content is a collision. An unsupported operation is preserved without changing materialized state.

Identity and presence

`PeerIdentity` contains a stable peer ID, the actor ID that connects presentation to authored operations, optional display name/color/avatar, and extensible attributes. `PeerPresence` adds a connection-scoped session, monotonic sequence, state, selections, generic focus metadata, timestamp, and TTL.

Receivers retain the greatest sequence per session and remove expired or offline sessions. Presence grants no permissions. All presentation fields are untrusted, and every endpoint decides what to display.

Swift API

IDs and values

- `SpaceID`, `ActorID`, `EntityID`, `OperationID`, `SessionID`: strongly typed string IDs.
- `ProtocolToken`: extensible string representation used by `OperationAction`, `EntityKind`, and `Feature`.
- `EntityReference`: entity kind plus stable ID.
- `OperationStamp`: deterministic `(sequence, actorID, operationID)` ordering tuple.
- `Value`: Codable JSON null, bool, number, string, array, or object.

Operations and codecs

- `EditOperation`: immutable durable edit.
- `Producer`: optional app/library provenance used for diagnostics.
- `OperationEnvelope`: `editspace.operations` batch.
- `OperationCodec`: sorted-key JSON encoding and validated decoding.
- `CodecError`: wrong kind, newer envelope schema, or space mismatch.

Compatibility and reduction

- `CompatibilityPolicy`: supported schema, actions, entity kinds, and features.
- `CompatibilityProblem`: structured non-destructive incompatibility.
- `OperationLog`: idempotent append-only accepted and rejected operation sets.
- `FieldRegister`, `EntityState`, `SpaceState`: materialized CRDT data.
- `Materializer`: deterministic log reduction.

Application projection

- `Replica`: owns the private materialized convergence index for one space.
- `ReplicaBindings`: application-supplied create, update, delete, and link functions.
- `ReplicaMutationContext`: originating operation plus the entity's complete resolved fields and arguments.
- `ReplicaApplicationReport`: applied operation IDs and projection failures.

Applications can rebuild their renderer or model by attaching new bindings and calling `replay()`. This keeps `SpaceState` available for diagnostics and conformance without making it a second application-state contract.

Synchronization

- `SyncEngine`: imports operations and routes accepted IDs to peer/store queues.
- `SyncResult`: accepted, duplicate, and rejected operation IDs.
- `OperationTransport`: async byte transport adapter interface.
- `OperationStore`: async append/fetch durable-store adapter interface.

Collaboration presence

- `PeerIdentity`: stable presentation-to-author link.
- `PeerPresence`: expiring session state.
- `PresenceEnvelope` and `PresenceCodec`: wire representation.
- `PresenceRoster`: newest-record and TTL handling.

Typical Swift flow

```swift
let spaceID = SpaceID(rawValue: "scene")
let operation = EditOperation(
    spaceID: spaceID,
    actorID: ActorID(rawValue: "ada"),
    sequence: 1,
    action: .create,
    entity: .object,
    targetID: EntityID(rawValue: "cube"),
    fields: ["name": .string("Cube")]
)

let replica = Replica(
    spaceID: spaceID,
    bindings: ReplicaBindings(
        create: { context in appScene.create(context.reference, fields: context.
fields) },
        update: { context, fields, _ in appScene.update(context.reference, field
s: fields) },
        delete: { context in appScene.delete(context.reference) },
        setLink: { context, linked, isLinked in
            appScene.setLink(from: context.reference, to: linked, isLinked: isLi
nked)
        }
    )
)
replica.apply([operation])
```

Blender/Python implementation checklist

Implement the JSON Schemas first and validate against shared golden fixtures. Pe
rsist EditSpace entity IDs in Blender custom properties rather than object names
. Keep the operation log separate from Blender's evaluated dependency graph. App
ly remote scene changes on Blender's main thread. Treat presence as disposable U
I state. Compare decoded semantic operation content when detecting duplicates, a
nd never use local wall-clock time for merge order.

Security and limits

Network v1 is an explicitly unencrypted trusted-LAN protocol for minimum cross-p
latform interoperability. Network v2 adds stable cryptographic peer identity, ex
plicit channel participation, durable operation attestations, and end-to-end enc
rypted drawing/session channels. Relay-only peers route opaque envelopes by rand
om channel identifier without receiving channel keys. Group-key admission and ro

tation should use a reviewed protocol rather than custom cryptography; Messaging Layer Security is the leading candidate pending constrained-link prototypes.

V2 implementations should bind authenticated connections to peer identity, keep presentation identity separate from cryptographic identity, sanitize user-facing strings and URLs, cap message size, nesting, operation count, dependency count and metadata sizes, and reject non-finite numbers. Presence is advisory and never conveys authorization. See the pinned protocol's networking specification for v1 LAN requirements and provisional v2 secure-LAN, offline-mesh, remote, and spatial profiles.

Versioning

Optional members may be added compatibly. Required new semantics use feature tokens. Incompatible representation changes increment the appropriate schema version. Extension tokens should use a collision-resistant namespace.

Further reference

The repository README contains Swift examples and API tables. The pinned `Protocol/` submodule contains the normative specification, JSON Schemas, and shared conformance vectors. The Xcode project generates symbol-level documentation from the public Swift doc comments and DocC catalog.